DATAWISE AI

DataWise AI: Architecture Report

In-Depth Technical Blueprint of Multi-Agent LangGraph Systems, State Machine & Resilience

Document Type:	System Architecture Specification
Target Audience:	Chief Architects, Principal Engineers, Security Reviewers
Document Version:	v1.0.0 (Engineering Release)
Date of Issue:	October 27, 2026
Repository Status:	Verified Codebase & Production Architecture
Orchestration:	LangGraph 16-Agent Autonomous Architecture

Architectural Foundation: Deterministic ML execution engine paired with autonomous multi-agent reasoning, strict data leakage boundaries, and human-in-the-loop validation.

1. Executive Architectural Blueprint

DataWise AI addresses the challenge of building dependable, enterprise-ready autonomous AI systems by establishing a decoupled, event-driven architecture. The core tenet is the absolute separation of non-deterministic cognitive reasoning (LLMs) from deterministic mathematical computation (scikit-learn / pandas).

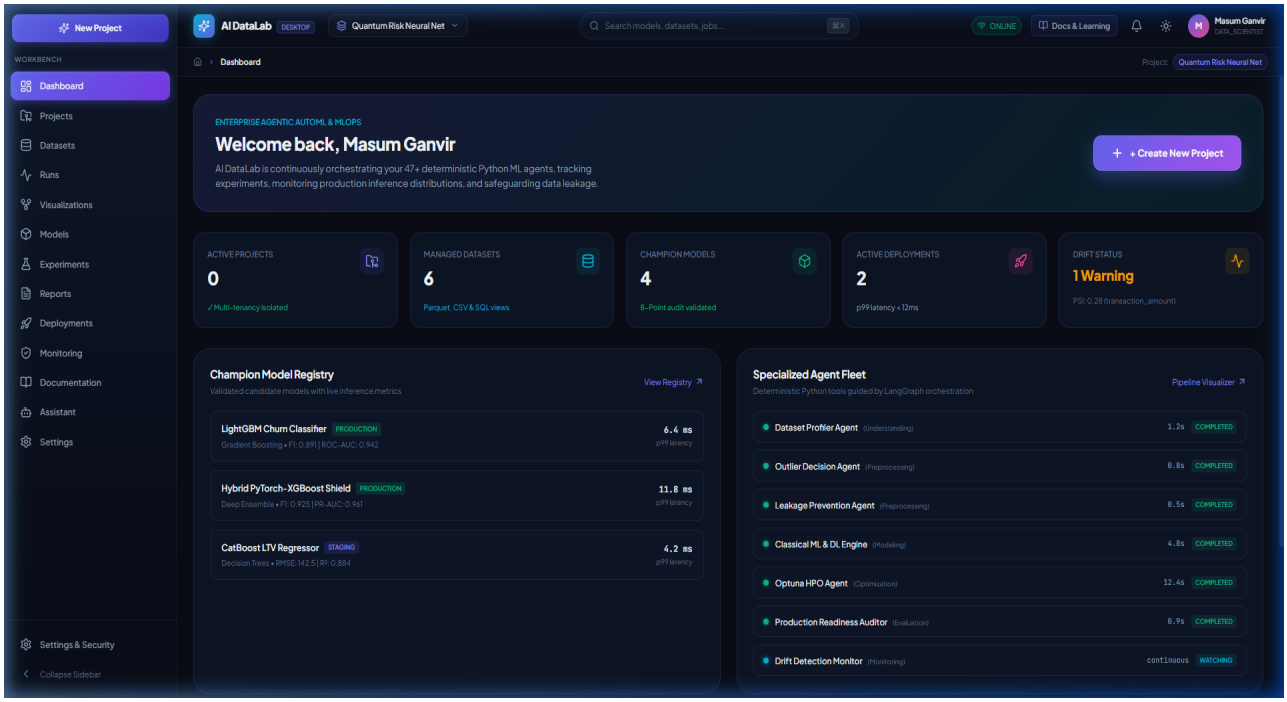


Figure: System Architectural View: Multi-Project Workspace & Real-Time Tracking

2. Component Topology & Data Flow Boundaries

The platform is partitioned into seven distinct runtime subsystems:

Subsystem	Runtime Process	Protocol	Security Context
Subsystem	Runtime Process	Communication Protocol	Security Boundary
Client UI	React 18 SPA (Browser)	HTTPS / WSS (WebSocket)	Clerk / JWT Bearer, CORS Restricted
API Gateway	FastAPI / Uvicorn	Async ASGI Event Loop	Rate Limiting, Pydantic Request Validation
Agent Graphs	LangGraph StateGraph	In-Process Asyncio	State Immutability, Step Gating
ML Execution	Deterministic Workers	Direct Python API	Memory Limits, Sandbox Validation
Relational DB	PostgreSQL / SQLite	SQLAlchemy 2.0 Async	Tenant Isolation, Row-Level Checks
Object Store	MinIO S3 / Local Disk	Boto3 / S3 REST API	Presigned URLs, Private Bucket ACLs
Cache & State	Redis Key-Value	Redis Async Protocol	Sliding Window Limiter, Session Token Store

3. LangGraph Workflow Graph Topologies

DataWise AI implements a hierarchical graph architecture managed by `backend/graphs/master_graph.py`:

- **Ingestion Subgraph:** File integrity check → Schema inference → Preview generation.
- **Data Analysis Subgraph:** Profiling → Quality scoring → Outlier detection → Correlation analysis.
- **Preprocessing Subgraph:** Missing value imputation → Encoding → Feature scaling → Leakage audit.
- **Modeling Subgraph:** Strategy recommendation → Baseline training → Optuna HPO → Final test scoring.
- **Artifact Subgraph:** Pipeline serialization → Jupyter notebook creation → PDF generation.

4. 18-Module Resilience & Self-Healing Architecture

Located in `backend/recovery/`, the resilience system safeguards operations against network degradation, LLM quota limits, memory exhaustion, and invalid data inputs. It provides circuit breakers, exponential backoff, state checkpoint restoration, and graceful fallback to local heuristic models if remote LLMs become unreachable.